KONZEPT

koje

Isolierte Arbeitskontexte unter Hyprland

Jedes Projekt in seiner eigenen Koje – isoliert, farbig markiert, auf Tastendruck da und wieder weg.



Autor Michael P. Jung

Stand 13. Oktober 2026

Technik Rust · Podman · Hyprland

ELEVATOR PITCH

Schluss mit dem Fenster-Chaos.

Wer an mehreren Projekten gleichzeitig arbeitet, verliert schnell den Überblick: Editoren, Browser, Terminals und Dokumente aller Projekte liegen durcheinander, Links öffnen sich im falschen Fenster, und Projekte, an denen gerade niemand arbeitet, belegen trotzdem Speicher.

koje trennt die Projekte: Jedes wird in einem eigenen, isolierten **Kontext** bearbeitet – seiner Koje. Ein Kontext ist ein Container, in dem alle Programme für dieses Projekt laufen, samt eigener Dev Containers; die Projekte sehen sich gegenseitig nicht. Auf dem Desktop trägt jedes Fenster die Farbe seines Kontexts. Benannte Layouts ordnen die Fenster per Tastendruck neu, und mit einem Befehl wird ein Kontext schlafen gelegt oder genau so wiederhergestellt, wie man es verlassen hat. Die Programme selbst bleiben unverändert.

Isoliert

Jedes Projekt in seinem eigenen Kontext – mit eigenem Browser, eigenem Podman und ohne Namenskonflikte.

Geordnet

Kontextfarbe an jedem Fensterrahmen, Layouts statt Workspace-Suche, Overlays für E-Mail und Chat.

Sparsam

Kontexte pausieren oder stoppen, Speicherlimits mit Warnung – und alles kommt zurück, wie es war.

Für wen	Entwickler, die unter Linux an mehreren Projekten parallel arbeiten
In einem Satz	Jedes Projekt in seiner eigenen Koje – isoliert, farbig markiert, auf Tastendruck da und wieder weg
Technik	Rust-Werkzeug mit Hintergrunddienst, Podman in Podman, Hyprland als Desktop

koje – Isolierte Arbeitskontexte unter Hyprland

Stand: 13. Oktober 2026 · Michael P. Jung

Kurzfassung: Mit `koje` wird jedes Projekt in einem eigenen, isolierten **Kontext** bearbeitet: einem Podman-Container, in dem alle Programme für dieses Projekt laufen – einschließlich eines eigenen Podman für die Dev Containers. Die Projekte sind dadurch voneinander getrennt und sehen sich gegenseitig nicht. Hyprland zeigt die Fenster aller Kontexte auf einem Desktop an und färbt jeden Fensterrahmen in der Farbe seines Kontexts. Innerhalb eines Kontexts wechselt man zwischen benannten **Layouts**; nicht benötigte Fenster liegen in einer Ablage in der Leiste. Kontextübergreifende Programme wie E-Mail oder Chat erscheinen in einblendbaren Overlays. `koje` startet und stoppt Kontexte als Einheit, stellt ihre Fenster wieder her, begrenzt ihren Speicher und warnt vor dem Limit. Die Programme selbst müssen dafür nicht angepasst werden.

Ziele

Ziel	Umsetzung
Fenster eines Kontexts gehören zusammen und sind als solche erkennbar	Ein Kontext = ein Container, ein Hyprland-Workspace, eine Rahmenfarbe
Links und Dateien öffnen im richtigen Kontext	Jeder Container hat seine eigenen Standardprogramme; was im Kontext geöffnet wird, bleibt dort
Kontexte sind voneinander isoliert	Eigene Namensräume für Prozesse, Netzwerk, IPC und Dateisystem; eigenes Podman pro Kontext
Inaktive Kontexte kosten nichts	Kontexte als Einheit stoppen oder pausieren; Speicherlimits pro Kontext
Die Arbeitsumgebung lässt sich wiederherstellen	Zustände und Layouts; laufender Absturzschutz
Keine Anpassung der Programme	Programme laufen im Container wie auf einem normalen System; optionale Integrationen für besseres Starten und Wiederherstellen

Grundprinzip

Kein Desktop kennt Einstellungen pro Workspace – Standardbrowser und Dateizuordnungen (XDG) gelten für die ganze Sitzung. Deshalb isoliert `koje` nicht den Workspace, sondern die **Arbeit daran**: Jeder Kontext ist ein Container mit eigenem Home-Verzeichnis und eigenen Standardprogrammen. Ein Link, der im Kontext geöffnet wird, landet automatisch beim Browser dieses Kontexts – ohne Profile, Wrapper oder Umgebungsvariablen.

Begriffe: Projekte werden in Kontexten isoliert – üblicherweise ein Projekt pro Kontext, bei zusammengehörigen Projekten (etwa Frontend und Backend eines Kunden) auch mehrere. `koje` selbst kennt dabei nur **Kontexte**: Die Projekte bindet ein Kontext einfach als Ordner ein; ein eigenes Projekt-Objekt gibt es in `koje` nicht. **Container** bezeichnet nur die technische Umsetzung (Kontext-Container, Dev Container), **koje** nur das Werkzeug.

Grafische Programme sprechen direkt mit dem Wayland-Compositor (Hyprland), über den Wayland-Socket. `koje` reicht diesen Socket in den Container durch; deshalb erscheinen Container-Fenster wie gewöhnliche Fenster auf dem Desktop. Alles Übrige – Farbe, Platzierung, Layouts – erledigt `koje` von außen über die Schnittstelle von Hyprland. Die Programme merken davon nichts.

Architektur



Beide Kontexte enthalten ein Compose-Projekt namens `backend` – dank eigenem Podman pro Kontext entstehen keine Namenskonflikte.

Kontext-Container

`koje` erzeugt die Kontext-Container direkt mit Podman. Jeder Container bekommt:

- ein eigenes Home-Volume und eigene Namensräume für Prozesse, Netzwerk und IPC (Standard bei Podman),
- den Wayland-Socket, den PipeWire-Socket für Audio und die Grafikkarte (`/dev/dri`),
- einen **gefilterten** D-Bus-Zugang über einen Proxy, wie ihn auch Flatpak nutzt: Benachrichtigungen und Dateiauswahl-Dialoge sind erlaubt, das Öffnen von Links auf dem Host ausdrücklich nicht – sonst würden Links den Kontext verlassen,
- ein Label mit dem Kontextnamen, über das `kojed` Prozesse und Fenster eindeutig zuordnet,
- Speicherlimits (siehe Abschnitt Speicher).

Podman in Podman

Jeder Kontext bringt sein eigenes rootless Podman mit. Dev Containers und Compose-Projekte laufen darin und sind für andere Kontexte und den Host unsichtbar. Gleichnamige Compose-Projekte in verschiedenen Kontexten kollidieren nicht.

Thema	Umsetzung
Image-Speicher innen	Eigenes Volume pro Kontext (verschachteltes overlayfs auf overlayfs ist nicht möglich)
Gemeinsame Images	Häufige Basis-Images über einen schreibgeschützten, gemeinsamen Image-Speicher (`additionalimagestores` in `storage.conf`) bereitstellen, statt sie pro Kontext erneut zu laden
Voraussetzungen	Eigener Bereich an Sub-UIDs und -GIDs im Container, Zugriff auf `/dev/fuse`, passende Storage-Konfiguration

Zu prüfen: Die Einrichtung von verschachteltem rootless Podman und die Nutzung eines gemeinsamen Image-Speichers aus dem inneren Podman einmal an einem Test-Kontext durchspielen und als Vorlage für das Basis-Image festhalten. Quelle

Sicherheit unter Wayland

Wayland trennt Programme grundsätzlich: Ein Fenster kann weder Eingaben noch Inhalte anderer Fenster lesen. Ausnahmen sind privilegierte Protokolle, etwa für Bildschirmaufnahmen. Hyprland bringt ein Berechtigungssystem mit, über das sich solche Zugriffe einschränken lassen; `koje` sollte es so konfigurieren, dass Programme aus Kontexten keine Bildschirmaufnahmen anderer Fenster anfertigen können. Quelle

Speicher

Was zwischen Kontexten geteilt wird

Programmcode und Bibliotheken werden vom Kernel als Dateien in den Speicher eingeblendet. Prozesse, die **dieselbe Datei** nutzen, teilen sich deren Speicherseiten – auch über Container hinweg. Voraussetzung ist, dass es auf dem Host tatsächlich dieselbe Datei ist: Alle Kontexte nutzen dasselbe Basis-Image, und Podman verwendet das native overlayfs des Kernels (rootless seit Kernel 5.13, bei aktuellen Systemen Standard).

Speicherart	Geteilt?	Anmerkung
Programmcode, Bibliotheken	Ja	Wenn aus derselben Image-Schicht; ein im Kontext nachinstalliertes Programm ist eine eigene Datei und wird nicht geteilt
Privater Speicher (Heap, JavaScript, Caches, Grafikpuffer)	Nein	Bei Browsern und Editoren der weitaus größte Anteil
Images der Dev Containers	Teilweise	Nur über den gemeinsamen, schreibgeschützten Image-Speicher
Dateicache der eingebundenen Ordner	–	Pro Datei; jeder Kontext arbeitet ohnehin an eigenen Dateien

Ein Browser pro Kontext kostet deshalb kaum mehr als parallel laufende Browser-Profile: Auch ein zweites Profil startet eine eigene Prozessfamilie mit eigenem privatem Speicher. Der Programmcode wird in beiden Fällen geteilt, solange er aus dem gemeinsamen Basis-Image stammt. Häufig genutzte Programme gehören daher ins Basis-Image, statt pro Kontext installiert zu werden.

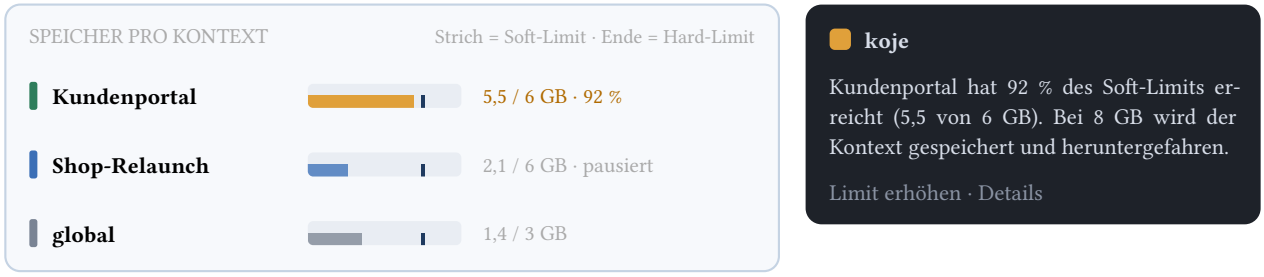
> **Hinweis zur Messung:** Geteilte Seiten rechnet der Kernel dem Kontext zu, der sie zuerst geladen hat. Die Speicheranzeige pro Kontext ist deshalb leicht ungenau; für Warnungen und Limits ist das unerheblich.

Limits pro Kontext

Grenze	Verhalten	Umsetzung
Soft-Limit	Warnung bei 90 % und bei 100 %; der Kontext läuft weiter	`kojed` überwacht den Speicherverbrauch der cgroup und sendet eine Desktop-Benachrichtigung
Hard-Limit	`koje` fährt den Kontext geordnet herunter – der Stand wird in `default` gespeichert – und meldet das	Überwachung durch `kojed`; Herunterfahren wie `koje down`
Sicherheitsnetz	Der Kernel greift ein, falls das geordnete Herunterfahren nicht rechtzeitig gelingt; betroffen ist nur dieser Kontext	cgroup-Grenze `memory.max`, etwas oberhalb des Hard-Limits

Limits lassen sich im laufenden Betrieb ändern (`koje limit`, intern über `podman update`). Ohne Angabe gelten Standardwerte aus der globalen Konfiguration.

Mockup: Speicherübersicht und Warnung



Links die Übersicht im Panel, rechts die Desktop-Benachrichtigung beim Erreichen von 90 %.

Desktop: Hyprland

Hyprland läuft komplett schwebend: Fenster werden frei platziert, nie automatisch angeordnet. Damit bleibt klassisches Arbeiten erhalten, und `koje` bekommt eine offene Schnittstelle zum Lesen und Steuern aller Fenster – Prozess-ID, Position, Größe, Workspace – sowie einen Ereignis-Socket, der jedes neue Fenster meldet. Quelle

Grundkonfiguration

```
# alle Fenster schwebend – Tiling ist damit faktisch aus
windowrule = float, class:.*
# Super + linke Maustaste: verschieben; Super + mittlere Maustaste: Größe ändern
bindm = SUPER, mouse:272, movewindow
bindm = SUPER, mouse:274, resizewindow
```

Die Konfigurationssyntax von Hyprland ändert sich zwischen Versionen öfter; das Beispiel vor dem Einsatz gegen die installierte Version prüfen. Leiste, Benachrichtigungen und Bildschirmsperre werden aus Einzelteilen zusammengestellt (etwa Waybar).

Kontextfarbe am Fensterrahmen

`kojed` erhält von Hyprland jedes neue Fenster, ermittelt über Prozess-ID und cgroup den zugehörigen Kontext und setzt Farbe und Stärke des Rahmens – für aktive und inaktive Fenster getrennt, auch als Verlauf. So trägt **jedes** Fenster eines Kontexts dessen Farbe, unabhängig vom Programm und ohne dessen Einstellungen anzufassen.

Workspaces

Ein Kontext belegt genau einen Hyprland-Workspace pro Monitor. Mehrere Arbeitsbereiche innerhalb eines Kontexts sind **Layouts** (eigener Abschnitt). Der Wechsel in einen anderen Kontext – per Menü oder `koje focus` – holt dessen Workspace mit dem zuletzt aktiven Layout nach vorn.

Einrasten

Art	Umsetzung	Stand
Kantenmagnetismus	Hyprland-Option für schwebende Fenster: Beim Ziehen rasten sie am Bildschirmrand und an Nachbarfenstern ein	eingebaut
Zonen per Taste	Raster pro Monitor in der `koje`-Konfiguration; `koje snap` setzt das aktive Fenster pixelgenau in eine Zone, ausgelöst per Tastenkombination	`koje`
Ziehen in Zonen mit Vorschau	Über die Schnittstelle nicht möglich; bräuchte ein Hyprland-Plugin	später

```
# ~/.config/koje/config.toml – Zonen für den 5K-Monitor (5120 × 2160)
[zones.DP-1]
links  = { x = 0,    y = 0, w = 1700, h = 2160 }
```

```
mitte  = { x = 1700, y = 0, w = 2100, h = 2160 }
rechts = { x = 3800, y = 0, w = 900,  h = 2160 }
chat   = { x = 4700, y = 0, w = 420,  h = 2160 }
```

Overlays

Kontextübergreifende Programme – E-Mail, Kalender, Chat – laufen im nicht isolierten Kontext `global`. Sie liegen in **Special Workspaces** von Hyprland: benannte Ebenen, die per Tastendruck über dem aktuellen Workspace ein- und wieder ausgeblendet werden.

- **Schnell:** Ein- und Ausblenden ist ein Tastendruck, der Workspace darunter bleibt unverändert.
- **Anheften:** `koje pin` holt das aktive Fenster aus dem Overlay auf den aktuellen Workspace und heftet es an – es bleibt sichtbar, auch wenn das Overlay ausgeblendet oder der Kontext gewechselt wird. So lässt sich eine Nachricht schreiben, während der Kontext daneben liegt. `koje unpin` schickt es zurück.

```
# ~/.config/koje/config.toml
[overlay.mail]
key  = "SUPER+M"
apps = ["mail", "calendar"]    # Integrationen oder Startbefehle

[overlay.chat]
key  = "SUPER+C"
apps = ["chat"]
```

Dateien zwischen Kontexten

Kopieren und Ziehen laufen über den Compositor, nicht über das Dateisystem. Bei Text und Bildern wird der Inhalt übertragen – das klappt immer. Bei Dateien wird meist nur der **Pfad** übergeben; das Zielprogramm muss die Datei selbst öffnen können.

Richtung	Ergebnis
Text, Bilder (Zwischenablage)	Funktioniert immer, auch zwischen Kontexten
Kontext ⟶ Overlay (`global`)	Funktioniert, wenn der Pfad im Container und auf dem Host gleich ist: `koje` bindet Home-Volume und eingebundene Ordner unter demselben Pfad ein wie auf dem Host
Host ⟶ Kontext	Nur, wenn der Pfad im Container eingebunden ist
Kontext A ⟶ Kontext B	Scheitert – das ist die gewollte Trennung

Ein Austauschordner (etwa `~/Austausch`) wird in alle Kontexte unter demselben Pfad eingebunden – ein bewusstes, kleines Loch in der Trennung.

Layouts

Prinzip: Ein Layout ist eine benannte Anordnung der Fenster eines Kontexts – etwa `code`, `review` oder `docs`. Ein Fenster darf in mehreren Layouts vorkommen, jeweils mit eigener Position und Größe. Fenster, die im aktiven Layout nicht vorkommen, liegen in der **Ablage** und erscheinen als Symbol in der Leiste. Layouts ersetzen mehrere Workspaces pro Kontext.

Begriffe

Kontext „Kundenportal“ · Container, Farbe

Zustand `default` · im Alltag der einzige

Fensterbestand: Editor · Browser · Terminal · Dokument

Layouts: code ● · review · docs

weitere Zustände optional, etwa `fresh` oder `workshop`

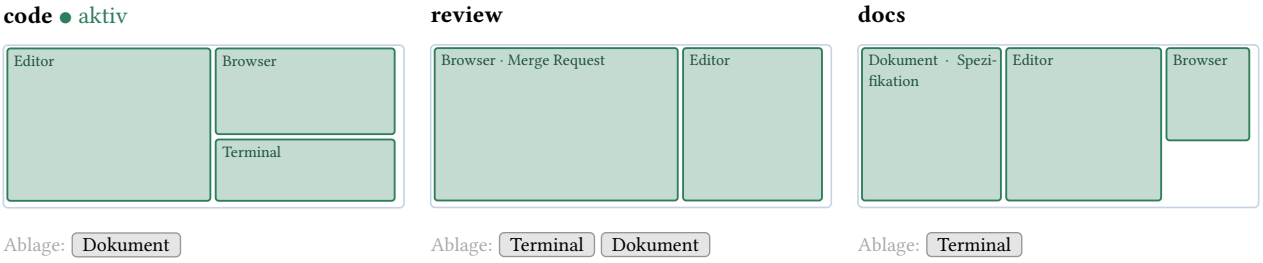
Begriff	Legt fest	Wechsel
Kontext	Container, Farbe, eingebundene Ordner, Fenster	sofort
Zustand	**Welche** Programme laufen – und deren Layouts	teuer, selten
Layout	**Wie** die laufenden Fenster angeordnet und welche sichtbar sind	Millisekunden
Ablage	Laufende Fenster, die im aktiven Layout fehlen	–

Ein Layoutwechsel

1. Fenster, die im Ziel-Layout fehlen, wandern in die Ablage – einen versteckten Special Workspace des Kontexts.
2. Fenster des Ziel-Layouts erhalten dessen Position und Größe; Fenster aus der Ablage werden dabei eingeblendet.
3. Der Fokus geht an das Fenster, das im Ziel-Layout zuletzt aktiv war.

Es wird nichts gestartet und nichts beendet. `koje` schickt alle Verschiebungen gebündelt in einem Aufruf an Hyprland; spürbar ist im Wesentlichen nur die Animation.

Mockup: dieselben Fenster in drei Layouts



Monitor im Seitenverhältnis 5120 × 2160. Der Editor ist in allen drei Layouts sichtbar, aber jedes Mal anders platziert; Terminal und Dokument erscheinen nur dort, wo sie gebraucht werden.

Ablage

Hyprland kennt kein echtes Minimieren; üblich ist ein versteckter Special Workspace – genau das ist die Ablage. Das Verschwinden und Erscheinen animiert Hyprland, etwa als Ausblenden oder Herauszoomen. Ein Flug-in-die-Leiste-Effekt bräuchte ein Hyprland-Plugin und ist für später vorgemerkt.

- **Leiste:** Das `koje`-Modul zeigt die Ablage des aktiven Kontexts als Symbole. Ein Klick holt ein Fenster ins aktive Layout – an die Stelle, an der es zuletzt in diesem Layout lag. Rechtsklick bietet „In Layout … holen“ und „Schließen“.
- **Tastatur:** `koje stash` (etwa Super + H) legt das aktive Fenster ab und nimmt es aus dem aktiven Layout.

Mockup: Leiste mit Kontext, Layouts, Ablage und Speicher

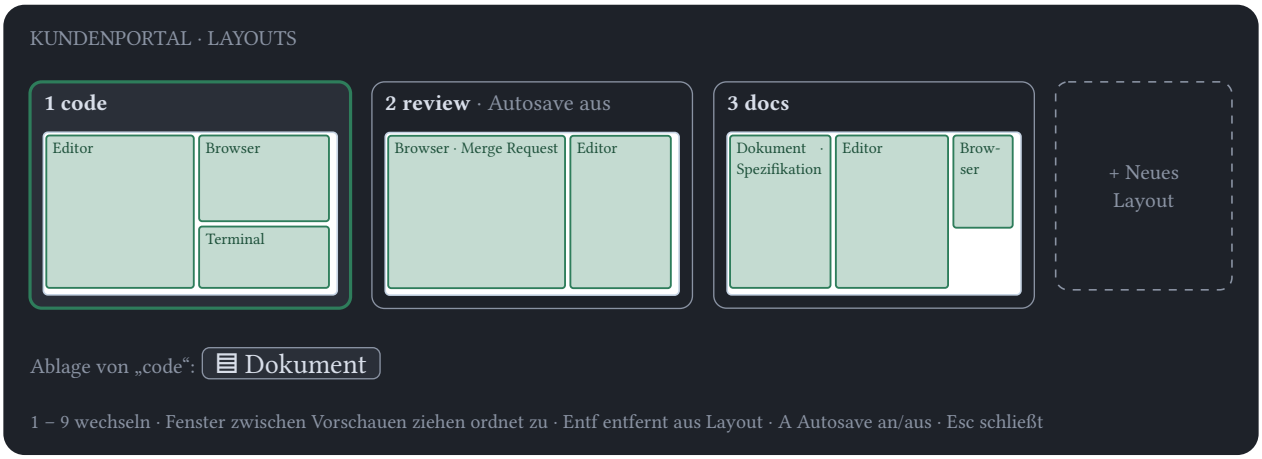


Links der aktive Kontext mit seinen Layouts (Super + 1 … 9 wechselt, Super + Tab blättert). Rechts die Ablage, der Speicherverbrauch – orange ab 90 % des Soft-Limits – und weitere Kontexte, hier ein pausierter. Klick auf die Kontext-Kapsel öffnet das Schnellstartmenü.

Layouts bearbeiten

- **Automatisch speichern** (Checkbox pro Layout, standardmäßig an): Was im aktiven Layout verschoben oder in der Größe geändert wird, übernimmt `koje` sofort in dieses Layout.
- **Ohne automatisches Speichern** wird ein Layout zur Vorlage: Änderungen gelten bis zum nächsten Wechsel; beim Zurückkehren erscheint wieder die gespeicherte Anordnung. `koje layout save` übernimmt den aktuellen Stand bewusst.
- **Neue Fenster** gehören zum aktiven Layout. **Geschlossene Fenster** verschwinden aus allen Layouts.
- **Neues Layout:** `koje layout new docs` startet leer, alle Fenster liegen zunächst in der Ablage; mit `--copy` übernimmt es die aktuelle Anordnung.

Mockup: Layout-Übersicht (Super + L)



Eingeblendet über dem Desktop. Ein Fenster in eine andere Vorschau zu ziehen, nimmt es in dieses Layout auf – so entsteht ein neues Layout, ohne Fenster einzeln zu verschieben.

Mockup: Panel, Reiter „Layouts“

Kundenportal — Zustände · **Layouts** · Fenster · Speicher

Fenster	code aktiv	review	docs	+ Layout
Editor · `editor`	●	●	●	
Browser · `browser`	●	●	●	
Terminal · `term` · Dev Container	●	○	○	
Dokument · `spec`	○	○	●	
Automatisch speichern	x	☐	x	

● sichtbar im Layout · ○ in der Ablage · Klick schaltet um · Doppelklick auf einen Layoutnamen wechselt dorthin

Die Fenster-IDs (`editor`, `term` …) vergibt `koje` automatisch aus Integration bzw. Programmname und einem Zähler; sie sind umbenennbar.

Wo Layouts gespeichert werden

Layouts sind Teil des Zustands. Ein Zustand speichert den Fensterbestand, alle Layouts mit Monitor, Position und Größe je Fenster sowie das aktive Layout. Ein Layout verweist auf konkrete Fenster – in einem anderen Zustand gäbe es diese womöglich nicht.

Für den Anwender bleibt es einfach: Im Alltag existiert nur `default`, die Layouts gehören also gefühlt zum Kontext. Ein weiterer Zustand ist ein Schnappschuss des ganzen Kontexts samt Layouts – nur für einen grundlegend anderen Arbeitsmodus.

```
# ~/.local/share/koje/kundenportal/states/default.toml
active_layout = "code"

windows = [
  { id = "editor",  integration = "editor",   path = "~/src/kundenportal" },
  { id = "browser", integration = "browser" },
  { id = "term",    integration = "terminal", where = "dev", cwd = "/workspace" },
  { id = "spec",    command = "evince docs/spezifikation.pdf" },
]

[layout.code.windows]          # Pixel, je Monitor
editor  = { monitor = "DP-1", x = 0,    y = 0,    w = 2660, h = 2160 }
browser = { monitor = "DP-1", x = 2660, y = 0,    w = 2460, h = 1240 }
term    = { monitor = "DP-1", x = 2660, y = 1240, w = 2460, h = 920 }

[layout.review]
autosave = false
[layout.review.windows]
browser = { monitor = "DP-1", x = 0,    y = 0, w = 3170, h = 2160 }
editor  = { monitor = "DP-1", x = 3170, y = 0, w = 1950, h = 2160 }
```

Layout oder Zustand?

Situation	Mittel
Dieselben Programme, andere Anordnung (Code schreiben, Review, Doku lesen)	Layout
Ein Programm kommt dazu oder fällt weg	Einfach öffnen oder schließen – landet automatisch in `default`
Grundlegend anderer Arbeitsmodus mit anderen Programmen (Präsentation, Workshop)	Zustand
Sauber neu anfangen	Zustand `fresh`

Integrationen

Programme brauchen keine Integration: Jedes Fenster wird erkannt, eingefärbt, in Layouts gespeichert und beim nächsten Start über seine ursprüngliche Kommandozeile wieder geöffnet. Integrationen sind **optional** und verbessern zwei Dinge – wie ein Programm gestartet wird und was beim Wiederherstellen gemerkt wird.

Integration	Leistung
`generic` (immer)	Erkennt Fenster über App-ID und Prozess; merkt sich die Kommandozeile
`terminal`	Merkt sich das Arbeitsverzeichnis der Shell; startet wahlweise im Kontext oder im Dev Container

Integration	Leistung
`editor`	Merkt sich den geöffneten Ordner; öffnet eingebundene Ordner gezielt
`devcontainer`	Startet den Dev Container im inneren Podman, bevor abhängige Fenster geöffnet werden
`browser`	Überlässt Tabs der Sitzungswiederherstellung des Browsers; nimmt Links von `koje open` entgegen

Eine Integration ist eine kleine Beschreibungsdatei; eigene lassen sich ergänzen:

```toml
# ~/.config/koje/integrations/editor.toml
name     = "editor"
match    = { app_id = ["code", "codium", "zed"] }   # Fenster erkennen
launch   = "{command} --new-window {path}"           # Start mit gemerkten Werten
remember = ["path"]                                  # was im Zustand landet
detect   = { path = "argv-last" }                    # woher der Wert stammt
```

Welche Programme ein Kontext enthält, entscheidet allein der Anwender bzw. das Basis-Image; Integrationen legen nur fest, **wie** `koje` mit ihnen umgeht.

Das Werkzeug: `koje`

Befehle

Befehl	Wirkung
`koje up <kontext> [-s <state>]`	Kontext starten, Fenster des Zustands öffnen und anordnen (Standard: `default`). Läuft er schon: in den Zustand wechseln
`koje up <kontext> --recover | --no-recover`	Nach einem Absturz den gesicherten Stand übernehmen bzw. verwerfen, ohne Rückfrage
`koje down <kontext> [-s <state>]`	Stand speichern (Standard: `default`), Fenster geordnet schließen, Container stoppen
`koje down <kontext> --no-save`	Herunterfahren ohne zu speichern; alle Zustände bleiben unverändert
`koje save <kontext> [-s <state>]`	Stand speichern, ohne herunterzufahren
`koje state list | cp | mv | rm`	Zustände verwalten, etwa `state cp kp default fresh`
`koje layout <name> | next | prev`	Layout wechseln
`koje layout new | save | autosave | list | mv | rm`	Layouts verwalten
`koje stash` / `koje unstash <fenster>`	Fenster in die Ablage legen bzw. ins aktive Layout holen
`koje focus <kontext>`	Zum Kontext wechseln
`koje snap <zone>`	Aktives Fenster in eine Zone setzen
`koje overlay <name>` / `koje pin` / `koje unpin`	Overlay umschalten; Fenster anheften bzw. zurückschicken
`koje sleep | wake <kontext>`	Kontext pausieren bzw. fortsetzen
`koje stats` / `koje limit <kontext>`	Speicher pro Kontext anzeigen; Limits ändern
`koje open <kontext> <url | datei>`	Link oder Datei im Kontext öffnen
`koje shell <kontext> [--dev]`	Shell im Kontext bzw. im Dev Container
`koje new | edit | rm <kontext>`, `koje list`	Kontexte anlegen, bearbeiten, entfernen, auflisten

Bei `down` schließen sich `-s` und `--no-save` aus. Herunterfahren entscheidet nur, **ob** gespeichert wird; Hochfahren nur, **was** geladen wird. Gelöscht wird ausschließlich mit `state rm`.

Zustände

Name	Art	Rolle
`default`	normal	Wird von `up` und `down` ohne `-s` gelesen bzw. geschrieben
`fresh`	normal	Gepflegter Startzustand, geladen mit `up -s fresh`
`empty`	reserviert	Virtuell und nur lesbar: Kontext starten, keine Fenster
beliebig	normal	Eigene Schnappschüsse für besondere Arbeitsmodi

Fallback beim Laden: Existiert der angefragte Zustand nicht, lädt `koje` erst `fresh`, dann `empty` und meldet das. Das passiert nur, wenn ein Zustand nie angelegt wurde. Die Rollennamen `default` und `fresh` sind in der `context.toml` umbenennbar; nur `empty` ist fest.

Zustand wechseln: Läuft der Kontext bereits, vergleicht `up -s` die offenen Fenster mit dem Ziel. Fenster in beiden bleiben offen und übernehmen die Layouts des Ziels; nur aktuell offene werden geordnet geschlossen (Programme fragen bei Ungespeichertem selbst nach); nur im Ziel vorhandene werden gestartet. Vorher wird wie bei `down` in `default` gespeichert, außer mit `--no-save`.

Absturzschutz

Läuft immer mit und braucht keine Einstellung:

1. `kojed` schreibt den aktuellen Stand nach jeder Änderung – kurz verzögert – nach `~/.local/state/koje/<kontext>/recovery.toml`, atomar über Temporärdatei und Umbenennen. Auch ein Stromausfall hinterlässt keine halb geschriebene Datei.
2. Bei `down` wird wie gewohnt gespeichert oder verworfen; danach wird die Datei gelöscht.
3. Findet `up` noch eine Wiederherstellungsdatei, fragt `koje`: „Kundenportal wurde nicht sauber beendet – Stand von 15:42 wiederherstellen?“

Wiederhergestellt werden Programme, Layouts und Fensterpositionen; Inhalte stellen die Programme selbst wieder her.

Pausieren

`koje sleep` friert den ganzen Kontext ein, einschließlich seiner Dev Containers: CPU-Last null, Fortsetzen in unter einer Sekunde. Der Speicher bleibt belegt, kann aber in den Swap ausgelagert werden. Die Fenster verschwinden vorher in der Ablage. Zu beachten: Netzwerkverbindungen können während der Pause ablaufen, und Hyprland darf eingefrorene Fenster nicht als „reagiert nicht“ melden (Einstellung prüfen).

Speicherorte

Pfad	Inhalt
`~/.config/koje/config.toml`	Globale Einstellungen: Zonen, Overlays, Standard-Limits
`~/.config/koje/contexts/<kontext>/context.toml`	Von Hand gepflegt: Name, Farbe, Image, Limits, Ordner
`~/.config/koje/integrations/*.toml`	Eigene Integrationen
`~/.local/share/koje/<kontext>/states/<name>.toml`	Zustände inklusive Layouts
`~/.local/state/koje/<kontext>/recovery.toml`	Absturzschutz; nach sauberem `down` gelöscht

Kontext-Konfiguration (Beispiel)

```toml
name   = "kundenportal"
prefix = "KP"
color  = "#2E7D5B"
image  = "koje/base:latest"         # gemeinsames Basis-Image

mounts = ["~/src/kundenportal", "~/Austausch"]   # gleicher Pfad wie auf dem Host

[limits]
memory_soft = "6G"                  # Warnung bei 90 % und 100 %
memory_hard = "8G"                  # geordnetes Herunterfahren

[states]                            # optional, Rollen umbenennen
```

```
default = "default"
fresh   = "fresh"
```

Aufbau

Baustein	Umsetzung
Aufteilung	Cargo-Workspace: `koje-core` (Logik), `kojed` (Hintergrunddienst), `koje` (Kommandozeile), `koje-gui` (Oberfläche)
Kommandozeile	`clap`; sich ausschließende Flags über Argumentgruppen
Konfiguration und Zustände	`serde` + `toml`; Pfade nach XDG über `directories`
Container	Podman zunächst über die Kommandozeile; später über dessen REST-Schnittstelle
Hyprland	Befehls- und Ereignis-Socket direkt ansprechen
Speicher	cgroup-Werte des Containers lesen; Benachrichtigungen über die Desktop-Schnittstelle
Clients ↔ `kojed`	D-Bus (`zbus`) oder eigener Unix-Socket
Oberfläche	iced, egui oder Slint; Overlay über das Layer-Shell-Protokoll

Oberfläche

Kommandozeile und Oberfläche sind gleichwertige Clients von `kojed`. Die Oberfläche entsteht in drei Stufen:

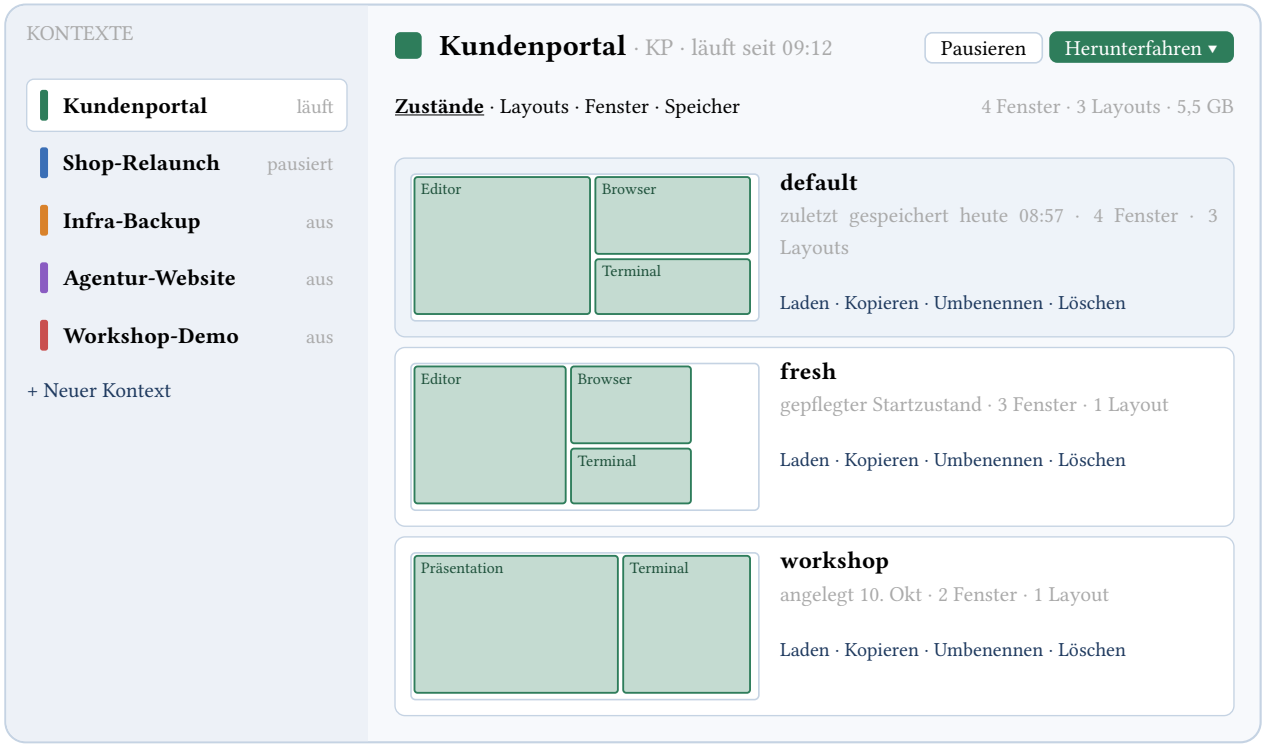
Stufe	Umsetzung
1 · Schnellstartmenü	Launcher (fuzzel oder rofi) mit Kontexten und Aktionen aus `koje`
2 · Leistenmodul	Waybar-Modul mit Kontext, Layouts, Ablage und Speicher (Mockup im Abschnitt Layouts)
3 · Panel	Eigenes Overlay in Rust über das Layer-Shell-Protokoll; Zustände, Layouts, Speicher

Mockup: Schnellstartmenü



Links: Kontexte mit Farbe und Laufzustand, Tippen filtert. Rechts: Aktionen für den gewählten Kontext; bei einem gestoppten steht oben „Starten“, darunter „Starten mit fresh“ und „Starten leer“.

Mockup: Panel, Reiter „Zustände“



Die Vorschau zeigt jeweils das aktive Layout eines Zustands im Seitenverhältnis 5120 × 2160.

Nächste Schritte

1. Hyprland einrichten: alle Fenster schwebend, Mausbelegung, Kantenmagnetismus, Overlays `mail` und `chat`; eine Woche im Alltag testen.
2. Basis-Image bauen und einen Kontext-Container von Hand starten: Wayland, PipeWire, Grafik, gefilterter D-Bus, Home-Volume.
3. Podman in Podman im Kontext einrichten, ein Compose-Projekt und einen Dev Container darin starten; gemeinsamen Image-Speicher testen.
4. Prüfen, dass sich Bibliotheken zwischen zwei Kontexten den Speicher teilen (gleiches Basis-Image, natives overlayfs).
5. Austauschordner einbinden und Ziehen von Dateien in ein Overlay-Programm testen.
6. Prototyp `kojed`: Fenster per Prozess-ID und cgroup zuordnen, Rahmenfarbe setzen, Speicher überwachen.
7. Prototyp Layoutwechsel: Fenster gebündelt verschieben, in die Ablage und zurück; Tempo und Animation beurteilen.
8. Erste Version von `koje up` und `koje down`: zunächst `empty` und `fresh`, danach `default` und Absturzschutz.
9. Schnellstartmenü als erste Oberfläche.

Quellen

- Hyprland IPC (Befehls- und Ereignis-Socket): wiki.hypr.land/IPC
- Hyprland Berechtigungen: wiki.hypr.land – Permissions
- Podman, zusätzliche Image-Speicher: Red Hat – Exploring additional image stores
- Minimieren unter Hyprland per Special Workspace: Waybar Issue 3374
- Podman-Checkpoints mit CRIU: Fedora Magazine – Podman Checkpoint

Anhang: Offene Punkte

Diese Fragen sind noch nicht entschieden oder müssen an einem Prototyp geprüft werden.

Container und Isolation

Punkt	Offene Frage
Podman in Podman	Welche Einstellungen braucht verschachteltes rootless Podman im Basis-Image (Sub-UIDs und -GIDs, `/dev/fuse`, Storage-Treiber)? Wie stabil läuft es im Alltag?
Gemeinsamer Image-Speicher	Lässt sich ein schreibgeschützter Image-Speicher (`additionalimagestores`) aus dem inneren Podman nutzen, und wie wird er aktualisiert?
D-Bus-Filter	Welche Schnittstellen erlaubt der Proxy genau? Gesetzt: Benachrichtigungen und Dateiauswahl ja, Öffnen von Links auf dem Host nein. Offen: Bildschirmfreigabe, Kalender, Schlüsselbund
Gleiche Pfade	Bindet `koje` Home-Volume und Ordner zuverlässig unter demselben Pfad wie auf dem Host ein, und klappt das Ziehen von Dateien in Overlay-Programme?
Kontext `global`	Ist es vertretbar, dass Overlay-Programme nicht isoliert laufen und alle eingebundenen Ordner sehen?
Berechtigungen in Hyprland	Wie werden Bildschirmaufnahmen für Programme aus Kontexten gesperrt, ohne gewollte Bildschirmfreigaben (etwa in Videokonferenzen) zu verhindern?

Speicher

Punkt	Offene Frage
Geteilte Bibliotheken	Teilen sich zwei Kontexte mit gleichem Basis-Image tatsächlich die Speicherseiten der Bibliotheken? Messung am Prototyp
Standard-Limits	Welche Soft- und Hard-Limits gelten ohne Angabe? Fester Wert oder Anteil am Arbeitsspeicher?
Hard-Limit	Wie lange darf das geordnete Herunterfahren dauern, bevor `koje` hart beendet? Wie groß ist der Abstand zur Kernel-Grenze `memory.max`?
Swap	Soll Swap pro Kontext begrenzt werden, und zählt er zum Limit?

Desktop und Layouts

Punkt	Offene Frage
Geschlossene Fenster	Verschwinden sie aus allen Layouts (derzeit vorgesehen), oder bleibt ein Platzhalter, den ein erneut gestartetes Programm wieder belegt?
Mehrere Monitore	Wie verhält sich ein Layout, wenn ein gespeicherter Monitor fehlt? Gilt ein Kontext-Workspace pro Monitor oder einer für alle?
Angeheftete Fenster	Gehört ein aus dem Overlay angeheftetes Fenster zum aktiven Layout, oder schwebt es über allen?

Punkt	Offene Frage
Zuordnung gleicher Programme	Wie zuverlässig lassen sich mehrere Fenster desselben Programms beim Wiederherstellen und beim Zustandswechsel zuordnen?
Pausierte Kontexte	Lässt sich die „reagiert nicht“-Meldung von Hyprland für eingefrorene Fenster gezielt unterdrücken?
Tempo des Layoutwechsels	Wirkt der Wechsel mit gebündelten Befehlen flüssig, und welche Animation passt?
Hyprland-Versionen	Wie fängt `koje` Änderungen an Konfigurationssyntax und Schnittstelle von Hyprland ab?

Werkzeug und Oberfläche

Punkt	Offene Frage
Integrationen	Reicht ein deklaratives Format, oder brauchen manche Integrationen ausführbare Hooks? Wie erkennt `terminal` das Arbeitsverzeichnis zuverlässig?
Fenster-IDs	Nach welcher Regel vergibt `koje` IDs, und wie bleiben sie über Neustarts stabil?
Schnittstelle zu `kojed`	D-Bus oder eigener Unix-Socket?
Podman-Anbindung	Ab wann lohnt der Wechsel von der Kommandozeile zur REST-Schnittstelle?
Oberflächen-Toolkit	iced, egui oder Slint für das Panel?
Sprache	Befehle, Hilfe und Meldungen auf Englisch, Deutsch oder beides?
Lizenz und Veröffentlichung	Wird `koje` Open Source, und unter welcher Lizenz?

Anhang: Ausblick

Ideen, die heute nicht umsetzbar oder bewusst zurückgestellt sind, aber später interessant werden können.

Echter Ruhezustand per CRIU

Das Pausieren hält den Speicher eines Kontexts belegt. Ein echter Ruhezustand würde den Kontext samt Arbeitsspeicher auf die Platte schreiben und später exakt so wiederherstellen – CPU **und** RAM wären frei. Podman unterstützt das über CRIU (`podman container checkpoint` und `restore`).

Heute scheitert das an zwei Punkten:

- **Nur mit Root:** Checkpoints funktionieren nur für Container, die als Root laufen; `koje` arbeitet rootless.
- **Offene Verbindungen nach außen:** CRIU kann Verbindungen zu Prozessen außerhalb des Containers kaum sichern. Jedes grafische Programm hängt am Wayland-Socket von Hyprland.

Interessant wird es, wenn CRIU rootless nutzbar wird. Ein erster Schritt könnten Kontexte oder Dev Containers ohne grafische Programme sein – etwa Datenbanken und Hintergrunddienste eines Projekts. Quelle

Weitere Ideen

Idee	Beschreibung
Flug-in-die-Leiste	Fenster fliegen beim Ablegen sichtbar in ihr Symbol in der Leiste; bräuchte ein Hyprland-Plugin
Zonen mit Vorschau	Fenster in vordefinierte Zonen ziehen, mit Vorschau während des Ziehens; bräuchte ein Hyprland-Plugin
Titelleisten in Kontextfarbe	Farbige Titelleisten über das offizielle Hyprland-Plugin, zusätzlich zum farbigen Rahmen